

Multi-Word Integer Operations and Types

Committee: ISO/IEC JTC1 SC22 WG21 SG6 Numerics

Document Number: P0104r1

Date: 2017-02-05

Authors: Lawrence Crowl

Reply To: Lawrence Crowl, Lawrence@Crowl.org

Abstract

We propose a set of operations and types for multi-word integer arithmetic.

Contents

[Introduction](#)

[Problem](#)

[Solution](#)

[Definition of Word](#)

[Multi-Int Types](#)

[Word Array Functions](#)

[Wording](#)

[??1 Multi-int arithmetic \[numbers.multiint\]](#)

[??1.1 Multi-int types \[numbers.multiint.types\]](#)

[??1.2 Multi-int member functions \[numbers.multiint.member\]](#)

[??1.3 Multi-int free functions \[numbers.multiint.free\]](#)

[??2 Word array arithmetic \[numbers.wordarray\]](#)

[Revisions](#)

Introduction

When the largest built-in integer type is too small for an application domain, one may want a larger type that behaves as though it were a built-in type.

Multi-word integer operations are a necessary component of many higher-level types, such as those that appear in other proposals.

Problem

There is no standard for such types. Libraries for multi-word and unbounded integers exist, but these were often designed to work in C and generally do not take full advantage of C++. More importantly, because they are not standard, it is hard to use them in code that interacts with foreign code.

The work for multi-word integer operations is tedious. For division it is also complicated. Furthermore, details and performance may vary between architectures. It is better to do that work once for each platform.

Solution

We propose a template for multi-word integer types that behave like built-in types. While built-in types often have insecure semantics, they are also efficient and well understood. Some programmers will prefer to use these types directly, but we also expect these types to be used as implementation tools for higher-level types.

We also propose a set of operations on word arrays. This low-level interface is intended as an implementation tool for higher-level operations and types, including those mentioned above. These operations can be built upon the wide operations presented in [P0103r1 Overflow-Detecting and Double-Wide Arithmetic Operations](#).

Definition of Word

A word is one of the types `single_sword` or `single_uword` as defined in [P0103r1 Overflow-Detecting and Double-Wide Arithmetic Operations](#).

Multi-Int Types

We provide multi-word versions of standard C++ integers. We call these multi-int types.

```
template<int words> multi_int;  
template<int words> multi_uint;
```

The following binary operators take two multi-int arguments of arbitrary size and sign. For the non-boolean results, the result type size is that of its largest argument. The result sign is unsigned if either argument is unsigned. There are also functions for a word with a multi-int.

```
~ * / % + - < > <= >= == != & | ^  
= *= /= %= += -= < > <= >= &= |= ^=
```

The following binary operators take a left-hand multi-int argument (of arbitrary size and sign) and a right-hand word argument. The result type is the type of the left-hand argument.

```
<< >> <<= >>=
```

There are conversions between the multi-int types and between them and the word types.

All types implicitly convert to `bool` via the normal rules.

Functions return by value. Because these types are potentially large, programmers should consider using compound assignment.

It seems reasonable to require `multi_int` to use a two's complement representation. However, the wording does not do so.

Word Array Functions

Word array functions provide mechanisms for efficient implementation of higher-level multi-word operations.

For each of the `split_...` functions in [P0103R1](#), there is are four functions of the following forms.

```
word fn( int length, word* result, const word* arg1, const word* arg2, ... )  
word fn( int length, word* result, const word* arg1, word arg2, ... )
```

These form would be used in the implementation of value-returning operations. The lengths of the arrays must be the same, so there is only a single length argument. The returned value is the 'carry out'.

```
word fn( int length, word* result_and_arg1, const word* arg2, ... )  
word fn( int length, word* result_and_arg1, word arg2, ... )
```

The first paramer is both the first argument and the result. This form would be used in the implementation of compound-assignment operations. The returned value is the 'carry out'.

These operations are not intended to provide complete multi-word operations, but rather to handle arrays with uniform operations. Higher-level operations then compose these operations into a complete operation. For example adding an n-word array to an m-word array, where $n > m$, would be accomplished by an m-word array plus array followed by an (n-m)-word array plus the word carried from the previous operation.

Wording

???.1 Multi-int arithmetic [numbers.multiint]

Add a new section:

Multi-int arithmetic is provided via two class templates and a set of functions mirroring the operations on built-in integral types.

In functions described in this section, implementations should enable the return value optimization.

???.1.1 Multi-int types [numbers.multiint.types]

Add a new section:

```
template <int words> multi_int;  
template <int words> multi_uint;
```

?.?.1.2 Multi-int member functions [numbers.multiint.member]

Add a new section:

The special member functions are as follows.

```
template <int words>  
multi_int<words>::multi_int( const multi_int& arg ) noexcept = default;  
  
template <int words>  
multi_uint<words>::multi_uint( const multi_uint& arg ) noexcept = default;  
  
template <int words>  
multi_int<words>&  
multi_int<words>::operator =( const multi_int& arg ) noexcept = default;  
  
template <int words>  
multi_uint<words>&  
multi_uint<words>::operator =( const multi_uint& arg ) noexcept = default;
```

Effects:

Constructs or assigns the object with the given argument.

Additional constructors are as follows.

```
template <int words>  
multi_int<words>::multi_int( single_sword arg ) noexcept;  
  
template <int words>  
explicit multi_int<words>::multi_int( single_uword arg ) noexcept;  
  
template <int words>  
multi_uint<words>::multi_uint( single_uword arg, ) noexcept;
```

Effects:

Constructs the object with the given argument.

Conversion operators are as follows.

```
template <int words>  
explicit multi_int<words>::operator single_sword() noexcept  
  
template <int words>  
explicit multi_uint<words>::operator single_uword() ) noexcept;
```

Returns:

The value of the object with any overly-large values handles as would built-in conversions.

```
template <int words>  
multi_int<words>::operator bool() noexcept  
  
template <int words>  
explicit multi_uint<words>::operator bool() ) noexcept;
```

Returns:

false if the value is zero, true otherwise.

For each assignment operator OP in $\{ = \ * = \ / = \ \% = \ + = \ - = \ \& = \ | = \ ^ = \}$, there shall be member functions as follows.

```
template <int words1> template <int words2>  
multi_int<words1>&  
multi_int<words1>::operator  $OP$ ( const multi_int<words2>& arg ) noexcept;  
  
template <int words1> template <int words2>  
multi_int<words1>&  
multi_int<words1>::operator  $OP$ ( const multi_uint<words2>& arg ) noexcept;  
  
template <int words1> template <int words2>  
multi_uint<words1>&  
multi_uint<words1>::operator  $OP$ ( const multi_int<words2>& arg ) noexcept;  
  
template <int words1> template <int words2>  
multi_uint<words1>&  
multi_uint<words1>::operator  $OP$ ( const multi_uint<words2>& arg ) noexcept;
```

Effects:

Assigns a value as would the corresponding built-in operations on built-in integral types.

Returns:

A reference to the object.

For each assignment operator OP in $\{ <= \ >= \}$, there shall be functions as follows.

```
template <int words>  
multi_uint<words>&  
operator  $OP$ ( const multi_int<words>& arg1, int arg2 ) noexcept;  
  
template <int words>  
multi_uint<words>&  
operator  $OP$ ( const multi_uint<words>& arg1, int arg2 ) noexcept;
```

Effects:

Assigns a value as would the corresponding built-in operations on built-in integral types.

Returns:

A reference to the object.

The attribute query functions are as follows.

```
template <int words>  
static constexpr size_t  
multi_int<words>::num_words() noexcept;  
  
template <int words>  
static constexpr size_t  
multi_uint<words>::num_words() noexcept;
```

Returns:

The number of words in the type.

```
template <int words>  
static constexpr size_t  
multi_int<words>::num_bits() noexcept;  
  
template <int words>  
static constexpr size_t  
multi_uint<words>::num_bits() noexcept;
```

Returns:

The total number of bits in the type. For `multi_int`, this number will include the sign bit.

??1.3 Multi-int free functions [`numbers.multiint.free`]

Add a new section:

For each unary operator OP in $\{ + \ - \ \sim \}$, there shall be a function as follows.

```
template <int words>  
multi_int<words>  
operator  $OP$ ( const multi_int<words1>& arg ) noexcept;
```

Effects:

Computes a result as would the corresponding built-in operations on built-in integral types.

Returns:

The result.

For each binary operator OP in $\{ * \ / \ \% \ + \ - \ \& \ | \ ^ \}$, there shall be functions as follows.

```
template <int words1, int words2>  
multi_int<std::max(words1,words2)>  
operator  $OP$ ( const multi_int<words1>& arg1, const multi_int<words2>& arg2 ) noexcept;
```

```
template <int words1, int words2>  
multi_uint<std::max(words1,words2)>  
operator  $OP$ ( const multi_uint<words1>& arg1, const multi_int<words2>& arg2 ) noexcept;
```

```
template <int words1, int words2>  
multi_uint<std::max(words1,words2)>  
operator  $OP$ ( const multi_int<words1>& arg1, const multi_uint<words2>& arg2 ) noexcept;
```

```
template <int words1, int words2>  
multi_uint<std::max(words1,words2)>  
operator  $OP$ ( const multi_uint<words1>& arg1, const multi_uint<words2>& arg2 ) noexcept;
```

```
template <int words>  
multi_int<words>  
operator  $OP$ ( const multi_int<words>& arg1, single_sword arg2 ) noexcept;
```

```
template <int words>  
multi_uint<words>  
operator  $OP$ ( const multi_uint<words1>& arg1, single_sword arg2 ) noexcept;
```

```
template <int words>  
multi_uint<words>  
operator  $OP$ ( const multi_int<words1>& arg1, single_uword arg2 ) noexcept;
```

```
template <int words>  
multi_uint<words>  
operator  $OP$ ( const multi_uint<words1>& arg1, single_uword arg2 ) noexcept;
```

```
template <int words>  
multi_int<words>  
operator  $OP$ ( single_sword arg1, const multi_int<words>& arg2 ) noexcept;
```

```
template <int words>  
multi_uint<words>
```

operator *OP*(single sword arg1, const multi uint<words1>& arg2) noexcept;

template <int words>

multi uint<words>

operator *OP*(single uword arg1, const multi int<words1>& arg2) noexcept;

template <int words>

multi uint<words>

operator *OP*(single uword arg1, const multi uint<words1>& arg2) noexcept;

Effects:

Computes a result as would the corresponding built-in operations on built-in integral types.

Returns:

The result.

For each binary operator *OP* in { < > <= >= == != }, there shall be functions as follows.

template <int words1, int words2>

bool

operator *OP*(const multi int<words1>& arg1, const multi int<words2>& arg2) noexcept;

template <int words1, int words2>

bool

operator *OP*(const multi uint<words1>& arg1, const multi int<words2>& arg2) noexcept;

template <int words1, int words2>

bool

operator *OP*(const multi int<words1>& arg1, const multi uint<words2>& arg2) noexcept;

template <int words1, int words2>

bool

operator *OP*(const multi uint<words1>& arg1, const multi uint<words2>& arg2) noexcept;

template <int words>

bool

operator *OP*(single sword arg1, const multi int<words>& arg2) noexcept;

template <int words>

bool

operator *OP*(const multi uint<words>& arg1, single sword arg2) noexcept;

template <int words>

bool

operator *OP*(single uword arg1, const multi uint<words1>& arg2) noexcept;

template <int words>

bool

operator *OP*(const multi uint<words1>& arg1, single uword arg2,) noexcept;

Effects:

Computes a result as would the corresponding built-in operations on built-in integral types.

Returns:

The result.

For each binary operator *OP* in { << >> }, there shall be functions as follows.

template <int words>

multi uint<words>&

operator *OP*(const multi int<words>& arg1, int arg2) noexcept;

```
template <int words>  
multi_uint<words>&  
operator OP( const multi_uint<words>& arg1, int arg2 ) noexcept;
```

Effects:

Computes a result as would the corresponding built-in operations on built-in integral types.

Returns:

The result.

???.2 Word array arithmetic [numbers.wordarray]

Section contents to be determined.

Revisions

This paper modifies [P0104R0](#) - 2015-09-27.

- Add Revisions section.
- Add Wording section.
- Remove Open Issues section.
- Clarify the distinction between multi-word and multi-int. The former is generic and the later is a specific set of types.
- Rename 'subarray' to 'word array' and substantially change the sections. They now reference [P0103R1](#).
- Move non-explanatory definitional statements from the Solution to the Wording.
- Change the definition of words to reflect. [P0103R1](#).
- Add `num_words` and `num_bits` type query functions.
- Add operations with a single word.
- Add conversions to and from a single word.